

Exercise 3: Enterprise Data Grid with Angular Material

Context: Basic loops using `@for` are effective for simple card arrangements but fall short when handling large enterprise datasets. For robust data management, applications require advanced column sorting, row pagination, and comprehensive ARIA accessibility features. Angular Material's `MatTable` package delivers these capabilities out of the box, utilizing a `MatTableDataSource` broker to wrap core arrays and feed layout properties straight to Material directives.

[!NOTE]

Step 1: Refactor the Enrollment List Component

Refactor your existing `EnrollmentListComponent` to replace the generic card layout with a structured Material grid.

Open `src/app/features/enrollment-list/enrollment-list.component.ts` and replace its entire content:

```
import { Component, viewChild, effect, inject } from
'@angular/core';
import { MatTableModule, MatTableDataSource } from
'@angular/material/table';
import { MatPaginatorModule, MatPaginator } from
'@angular/material/paginator';
import { MatSortModule, MatSort } from '@angular/material/sort';
import { EnrollmentStore } from '../../store/enrollment.store';
import { Enrollment } from '../../models/enrollment.model';

@Component({
  selector: 'tms-enrollment-list',
  standalone: true,
  imports: [MatTableModule, MatPaginatorModule, MatSortModule],
  templateUrl: './enrollment-list.component.html',
  styleUrls: ['./enrollment-list.component.scss']
})
export class EnrollmentListComponent {
  store = inject(EnrollmentStore);
  displayedColumns = ['studentName', 'courseName', 'status',
'actions'];

  // MatTableDataSource bridges our store data into Material's
  rendering pipeline
  dataSource = new MatTableDataSource<Enrollment>();

  // viewChild.required() is Angular's signal-based replacement for
  the legacy @ViewChild decorator.
  // These return responsive signals that update automatically as
  soon as the DOM queries resolve,
  // eliminating the need for the ngAfterViewInit lifecycle hook.
  readonly paginator = viewChild.required(MatPaginator);
  readonly sort = viewChild.required(MatSort);
```

```

    constructor() {
        // Effect 1: Push store entities into the Material data source
        whenever they change.
        // Fires automatically on mutations (approve, load, rollback)
        to re-render the view layer.
        effect(() => {
            this.dataSource.data = this.store.entities();
        });

        // Effect 2: Wire paginator and sort controls once Angular
        resolves the view queries.
        // Automatically handles execution as soon as the signals emit
        the bound template elements.
        effect(() => {
            this.dataSource.paginator = this.paginator();
            this.dataSource.sort = this.sort();
        });

        // Load enrollments on component creation with zero lifecycle
        hook dependencies
        this.store.loadEnrollments();
    }
}

```

[!NOTE]

Step 2: Build the Grid Template

Open `src/app/features/enrollment-list/enrollment-list.component.html` and replace its entire layout markup code:

```

<h2>Enrollment Records</h2>

@if (store.isLoading()) {
    <p>Loading enrollments...</p>
}

@if (store.error()) {
    <p class="error">{{ store.error() }}</p>
}

<table mat-table [dataSource]="dataSource" matSort class="mat-
elevation-z8">
    <!-- Student Name Column -->
    <ng-container matColumnDef="studentName">
        <th mat-header-cell *matHeaderCellDef mat-sort-
header>Student</th>
        <td mat-cell *matCellDef="let row">{{ row.studentName }}</td>
    </ng-container>

```

```

<!-- Course Name Column -->
<ng-container matColumnDef="courseName">
  <th mat-header-cell *matHeaderCellDef mat-sort-
header>Course</th>
  <td mat-cell *matCellDef="let row">{{ row.courseName }}</td>
</ng-container>

<!-- Status Column -->
<ng-container matColumnDef="status">
  <th mat-header-cell *matHeaderCellDef mat-sort-
header>Status</th>
  <td mat-cell *matCellDef="let row">
    <span class="status-badge"
[class]="row.status.toLowerCase()">{{ row.status }}</span>
  </td>
</ng-container>

<!-- Actions Column -->
<ng-container matColumnDef="actions">
  <th mat-header-cell *matHeaderCellDef>Actions</th>
  <td mat-cell *matCellDef="let row">
    @if (row.status === 'Pending') {
      <button
(click)="store.approveEnrollment(row.id)">Approve</button>
    }
  </td>
</ng-container>

<!-- Row definitions -->
<tr mat-header-row *matHeaderRowDef="displayedColumns"></tr>
<tr mat-row *matRowDef="let row; columns: displayedColumns;">
</tr>
</table>

<mat-paginator [pageSizeOptions]="[10, 25, 50]"
showFirstLastButtons></mat-paginator>

```

Why structural cell definitions match templates: The grid system continues to leverage asterisks syntax (**matHeaderCellDef*, **matCellDef*) instead of modern *@for* blocks because the data table architecture is fundamentally template-driven. Each column container acts as a repeatable structural blueprint that the grid framework instantiates contextually per row to manage accessibility, row generation, and internal virtual sorting vectors.

[!NOTE]

Step 3: Register the Enrollment Index Route

Open `src/app/app.routes.ts` and ensure your collection routing matches your feature navigation tables:

```
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: 'dashboard',
    loadComponent: () =>
      import('./features/instructor-dashboard/instructor-
dashboard.component')
        .then(m => m.InstructorDashboardComponent)
  },
  {
    path: 'enrollments',
    loadComponent: () =>
      import('./features/enrollment-list/enrollment-
list.component')
        .then(m => m.EnrollmentListComponent)
  },
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }
];
```

Exercise 3 Verification and Testing Checklist

1. **Initialize the Frontend Workspace:** Spin up your client server instance (`ng serve`) and navigate your browser window to `http://localhost:4200/enrollments`.
2. **Validate Active Data Sorting:** Click the **Student** header segment. Verify rows organize alphabetically. Toggle a second click to check inverse ordering. Repeat validation against **Course** and **Status** layout blocks.
3. **Validate Row Pagination Controls:** Ensure that data blocks scale smoothly by changing page sizing dropdown targets between 10, 25, and 50 configurations, navigating views using the pagination forward/back arrows.
4. **Verify Asynchronous Optimistic Actions:** Find a row item containing a "Pending" status and select **Approve**. Confirm that the element status updates immediately. If the separate instructor counter dashboard tab is open simultaneously, check that the global pending total drops immediately.
5. **Enforce Screen Accessibility Policies:** Use your keyboard `Tab` key to shift cursor focus directly inside the data grid headers. Verify that hitting `Enter` successfully executes sorting rules, ensuring text elements announce properly on standard screen-reader clients.